A Systematic Analysis of KV Cache Compression Policies for Long-Context Inference

Jiayue Xu
UC San Diego
jixu@ucsd.edu

Xuanwen Hua
UC San Diego
x2hua@ucsd.edu

Sijin Lyu
UC San Diego
s1lyu@ucsd.edu

Abstract

Running large language models on long inputs can be very expensive due to the significant memory and compute involved. A potential area to explore these costs further is shrinking it via KV cache compression, and in this project we compared a few simplified KV cache compression methods: Streaming style, H2O style, SnapKV style heuristics - all against a full-cache baseline under different cache budgets.

We measure task quality, latency, forward-pass time, compression overhead. We find that these methods greatly shrink KV cache size; 0.5 and 0.2 budgets compress to roughly half and fifth of full baseline. Quality is often still acceptable at 0.5 budget, but drops a bit more at 0.2. Speedup is much smaller, with only limited E2E gains observed mostly on long inputs. Overall, our experiments suggest that KV cache compression in our prototype is mainly about tradeoff between memory use and quality as opposed to get big latency wins.

Our code is available at `https://github.com/qweasdzxc-qwe/dsc291_efficientai_finalproject/tree/main`

1 Introduction

Large language models (LLMs) have shown strong performance on long-context tasks such as document understanding and reasoning. However, the computational cost of self-attention increases rapidly as the sequence length grows, making long-context inference expensive.

During autoregressive decoding, LLMs maintain a Key-Value (KV) cache that stores past token representations. Although KV caching improves efficiency, its memory usage grows linearly with sequence length, creating a new bottleneck.

Recent work proposes KV cache compression methods that retain only important tokens instead of the entire history. These methods introduce different strategies for selecting tokens, leading to trade-offs between efficiency and quality.

In this project, we aim to analyze KV cache compression policies under a unified framework and provide empirical insights into their performance.

2 Related Work

Prior studies have addressed the problem of improving long-context efficiency through efficient attention mechanisms as well as KV cache compression methods. Several works propose efficient transformers that reduce the quadratic cost of self-attention, allowing longer sequences without a large increase in computation Child et al. [2019], Beltagy et al. [2020], Zaheer et al. [2020]. However, these approaches usually require changes to the model architecture.

More recent work focuses on inference-time optimization through KV cache compression, where memory usage is reduced during autoregressive decoding without retraining the model. StreamingLLM proposes a streaming attention strategy that keeps recent tokens and important tokens in the cache Xiao et al. [2023]. PyramidKV introduces hierarchical compression to preserve multi-resolution context information Zhang et al. [2024]. SnapKV selects important tokens based on attention patterns, while H2O applies an importance-based eviction policy to remove less useful tokens during decoding Li et al. [2024], Zhang et al. [2023].

Although these methods show promising results, they are often evaluated under different experimental settings, which makes direct comparison difficult. A systematic evaluation using common benchmarks such as LongBench can provide a clearer understanding of the trade-offs between efficiency and quality Bai et al. [2023].

3 Problem Statement

We aim to answer the following questions:

- How do different KV cache compression methods affect retained KV cache size, forward-pass time, and end-to-end latency?
- How does the compression budget (1.0, 0.5, and 0.2) affect task quality across different tasks?
- What trade-offs arise between memory savings, latency, and task quality under a unified prototype benchmark?

Our goal is not to propose a new method, but to provide a systematic analysis.

4 Methodology

In this project, we study KV cache compression for long-context inference by implementing and comparing four decoding strategies under a unified framework: the full KV cache baseline, streaming-style compression, H2O-style importance-based compression, and SnapKV-style attention-based compression. All methods are implemented on top of the LLaMA-2-7B-Chat model using the HuggingFace Transformers library, and evaluated under identical decoding settings. Our focus is on inference-time optimization, and no additional training is performed.

4.1 Full KV Cache (Baseline)

The full-cache method follows standard autoregressive decoding without any compression. During generation, all past key and value tensors are retained and reused at every decoding step. This ensures that the model always has access to the complete context, which typically leads to the best task performance. However, the KV cache grows linearly with sequence length, resulting in high memory usage and increasing computational cost. This method serves as the reference baseline for all comparisons.

4.2 Streaming-Based Compression

The streaming method reduces memory usage by limiting the KV cache to a subset of tokens consisting of an initial set of tokens (often referred to as sink tokens) and a sliding window of the most recent tokens. Given a sequence of length N and a target cache size K, the method retains the first few tokens and the last $K - s$ tokens, where s denotes the number of sink tokens.

In our implementation, this selection is performed dynamically during decoding by recomputing the indices of tokens to retain and slicing the KV cache tensors accordingly. This approach significantly reduces the memory footprint by preventing attention over distant tokens. However, because all middle tokens are discarded, the model may lose important long-range dependencies, which can negatively affect performance on tasks that require global context.

4.3 H2O-Style Importance-Based Compression

The H2O-style method applies adaptive compression based on token importance rather than fixed positions. We estimate token importance using attention scores from the last transformer layer. Specifically, the importance of each token is computed by aggregating attention weights across all heads and query positions, resulting in a single score per token.

At each decoding step, the method retains a subset of tokens determined by a budget ratio r, such that only $K = \lfloor rN \rfloor$ tokens are kept. The retained tokens include the most recent tokens, which preserve recency information, as well as the top-ranked tokens from earlier positions based on their importance scores. This allows the model to selectively preserve information that is likely to be useful for future predictions.

Unlike a one-time compression strategy, our implementation applies this process iteratively during decoding. As a result, the KV cache is repeatedly updated using tensor operations such as sorting and indexing. While this improves flexibility in controlling the cache size, it also introduces additional computational overhead, which is reflected in our efficiency measurements.

4.4 SnapKV-Style Attention Window Compression

We also implement a simplified version of SnapKV that selects tokens based on recent attention patterns. Instead of computing importance over the entire sequence, this method focuses on a sliding window of recent queries. The attention scores within this window are averaged to produce a score for each token, which reflects its relevance to the most recent decoding steps.

Based on these scores, the method retains the top-K tokens under a given budget ratio. Compared to the H2O-style method, this approach avoids repeated importance recomputation and therefore introduces minimal compression overhead. In practice, this makes SnapKV more efficient while still capturing useful contextual information from recent attention dynamics.

4.5 Unified Compression Framework

All methods are implemented within a unified interface to ensure fair comparison. For each input prompt, the model first performs a prefill step to initialize the KV cache. During autoregressive decoding, each method optionally applies its corresponding compression strategy to the KV cache before generating the next token. Throughout this process, we record multiple efficiency metrics, including KV cache size, forward-pass time, compression overhead, and end-to-end latency.

We control the strength of compression using a budget ratio $r \in \{1.0, 0.5, 0.2\}$, where $r = 1.0$ corresponds to the full-cache baseline. This unified setup allows us to systematically analyze the trade-offs between memory usage, computational cost, and task performance across different KV cache compression strategies.

5 Experimental Setup

We run all experiments using LLaMA-2-7B-Chat in FP16 on a single A100 40GB GPU via Google Colab. We compare four decode settings: Full cache baseline, a streaming compression method, a h2o compression method, and a snapkv compression method. And for these compression methods, we test 2 budgets: 0.5 and 0.2, the full baseline use budget at 1.0. All methods are testing in the same prototype codebase.

We evaluate these methods on a sampled subset of LongBench tasks. Specifically, we use three representative tasks: NarrativeQA, HotpotQA, and GovReport, with 40 examples per task. To make these experiments tractable in our Colab-based environment, we truncate all long inputs to a fixed maximum input length of 4096, and use the same preprocessing and decoding settings across all methods.

For quality evaluation, we use task-specific metrics. NarrativeQA and HotpotQA are evaluated with F1, while GovReport is evaluated with Rouge-L. For efficiency, we report end-to-end latency, forward-pass time, compression overhead, and retained KV cache size in GB. We also compute speedup relative to the full-cache baseline in each task. We report retained KV cache size rather than peak GPU memory.

All experiments are run with the same model configuration and generation settings across the different methods - e.g. we use greedy decoding and the same maximum generation length for each task setting. Our goal is to compare the quality–efficiency trade-offs of different compression policies under controlled settings, rather than to demonstrate production-level performance.

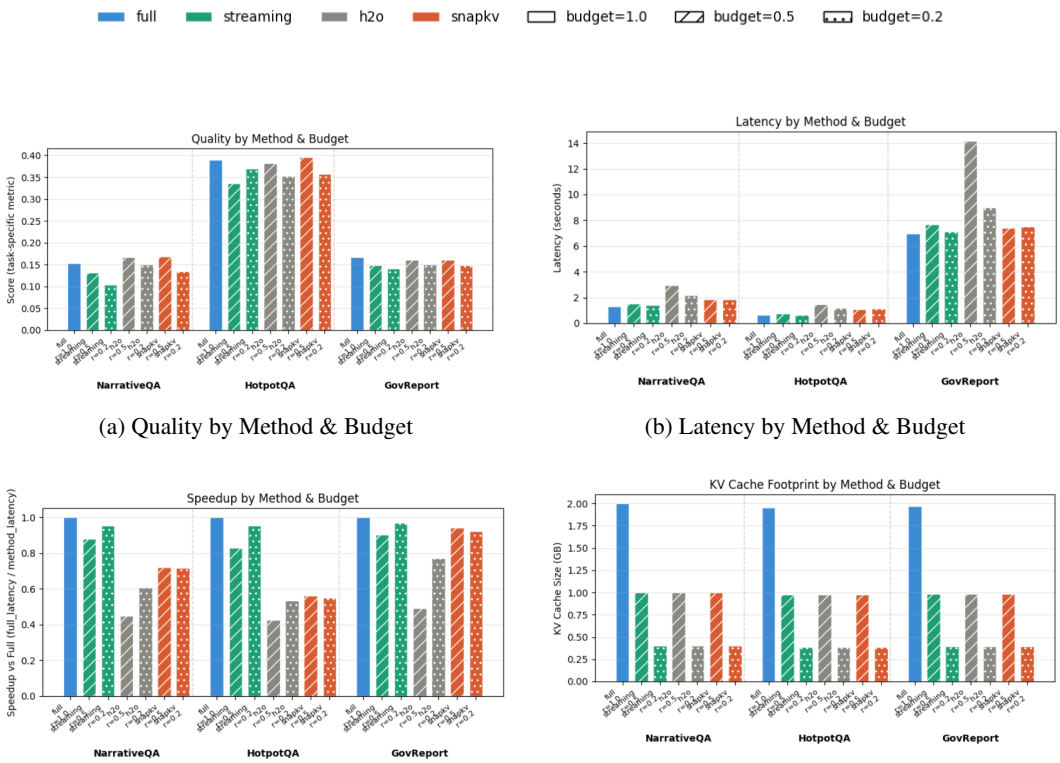
6 Results

Table 1: Quality and memory results on LongBench (40 samples per task).

Task	Method	Budget	Score	KV Cache (GB)	Reduction
NarrativeQA	Full	1.0	0.154	2.00	–
	Streaming	0.5	0.133	1.00	2.0×
	Streaming	0.2	0.104	0.40	5.0×
	H2O	0.5	0.168	1.00	2.0×
	H2O	0.2	0.151	0.40	5.0×
	SnapKV	0.5	0.170	1.00	2.0×
	SnapKV	0.2	0.135	0.40	5.0×
HotpotQA	Full	1.0	0.390	1.96	–
	Streaming	0.5	0.336	0.98	2.0×
	Streaming	0.2	0.371	0.39	5.0×
	H2O	0.5	0.382	0.98	2.0×
	H2O	0.2	0.353	0.39	5.0×
	SnapKV	0.5	0.396	0.98	2.0×
	SnapKV	0.2	0.358	0.39	5.0×
GovReport	Full	1.0	0.167	1.97	–
	Streaming	0.5	0.149	0.98	2.0×
	Streaming	0.2	0.141	0.39	5.0×
	H2O	0.5	0.161	0.98	2.0×
	H2O	0.2	0.151	0.39	5.0×
	SnapKV	0.5	0.161	0.98	2.0×
	SnapKV	0.2	0.149	0.39	5.0×

Table 2: Efficiency results on LongBench.

Task	Method	Budget	Latency (s)	Fwd (s)	Comp (s)	Speedup
NarrativeQA	Full	1.0	1.34	1.33	0.000	1.000
	Streaming	0.5	1.52	1.37	0.143	0.877
	Streaming	0.2	1.40	1.33	0.066	0.953
	H2O	0.5	2.97	2.43	0.293	0.451
	H2O	0.2	2.20	2.09	0.072	0.610
	SnapKV	0.5	1.85	1.84	0.005	0.723
	SnapKV	0.2	1.87	1.85	0.003	0.717
HotpotQA	Full	1.0	0.63	0.63	0.000	1.000
	Streaming	0.5	0.76	0.71	0.051	0.832
	Streaming	0.2	0.66	0.64	0.021	0.956
	H2O	0.5	1.48	1.32	0.081	0.428
	H2O	0.2	1.18	1.14	0.017	0.536
	SnapKV	0.5	1.12	1.12	0.005	0.564
	SnapKV	0.2	1.15	1.15	0.003	0.549
GovReport	Full	1.0	6.95	6.90	0.000	1.000
	Streaming	0.5	7.70	6.76	0.880	0.903
	Streaming	0.2	7.16	6.68	0.413	0.971
	H2O	0.5	14.16	10.89	1.838	0.491
	H2O	0.2	9.02	8.48	0.435	0.771
	SnapKV	0.5	7.39	7.33	0.005	0.941
	SnapKV	0.2	7.54	7.48	0.003	0.922



(a) Quality by Method & Budget

(b) Latency by Method & Budget

(c) Speedup by Method & Budget

(d) KV Cache Footprint by Method & Budget

Figure 1: Efficiency vs. quality tradeoff across three LongBench tasks

Tables 1 and 2 show that KV cache compression consistently retains a smaller cache size over the three tasks. Over full-cache baseline, a 0.5 budget reduces KV cache from 2.0 GB to 1.0 GB, with a 0.2 budget shrinking it to 0.4 GB. This trend is quite consistent across NarrativeQA, HotpotQA and GovReport: we're enforcing KV cache budgets as we desired.

In terms of quality, we find that 0.5 budget settings are reasonably stable, i.e. less variance between budgets than the 0.2 budget settings were shown to be. In HotpotQA, SnapKV with 0.5 budget reaches score of 0.396 (vs full baseline 0.390), and H2O with 0.5 budget is close behind at 0.382. In GovReport, both H2O and SnapKV with 0.5 budget were at 0.161 (vs. full baseline 0.167). Some tasks were less sensitive to compression than others, and so we suspect moderate compression is able to preserve nearly all of the quality of the original, while more aggressive compression leads to significantly larger degradation on tasks like GovReport and HotpotQA that require compression less: NarrativeQA appears to vary substantially across methods, and this task appears more sensitive to KV cache compression as well as small-sample effects.

For efficiency, we primarily see memory savings and not really a strong end-to-end speedup on our prototype for deep learning models. Overall latency does not beat the full-cache baseline reliably, especially after the compression is accounted for we suspect, but we do see modest improvement in forward-pass time on long-input settings, especially for Streaming on GovReport. Overall, Streaming looks like it has the clearest limited speed benefit on longer tasks; SnapKV achieves similar quality to H2O while having negligible compression overhead.

7 Analysis

Overall we find that the clearest benefit of KV cache compression in our prototype is memory. In all three tasks the retained KV cache size decreases quite clearly as the budget gets smaller and smaller. This indicates that the budget control in our implementation does what it is designed to do, and that all three compression schemes are working as intended to reduce cache footprint.

Another clear signal is that a 0.5 budget is in general less unpredictable as compared to a 0.2 budget in terms of task quality. This makes sense - a larger budget keeps more of the context around, while a smaller budget is more likely to purge tokens that are still useful for prediction. This difference is most visible for longer and more demanding tasks like GovReport. At the same time, a few cases show a 0.2 budget or a compressed setting beating out a larger budget or even the full baseline. We take this to be an example of sampled-subset variation and metric level noise, not that stronger compression is certainly better and that we should have always tried a smaller budget.

The speed benefit is more modest than the memory benefit - since we are reducing the size of KV cache the forward-pass cost also gets cheaper, but end-to-end latency also includes prefill time, compression, and framework-level runtime overhead. So not all improvements we see in forward time translate into clear total speedup. Specifically, we believe that there are three reasons for this lack of speedup. First, our compression logic implemented as Python-level tensor operations (indexing, sorting, concatenation) run at every step of decoding and are not fused into the GPU forward - for example, H2O's overhead is 1.84s on GovReport at 0.5 budget, almost 26 percent of the full baseline's total latency. Second, our inputs are short enough (capped at 4096 tokens) that self-attention on the full cache is already fast on an A100 and the compute savings from reducing the cache is minor relative to the overhead introduced. Lastly, in order to meaningfully gain latency from a smaller KV cache, you would need to have some kind of kernel-level sparse attention implementation (like those in vLLM or SGLang) that fuse the selection of tokens and the attention computation into a single GPU kernel and avoid the whole Python-side overhead.

The three methods have different trade-offs as well. Streaming has the clearest no-speed benefit longer task, but its quality can drop more aggressive compression settings, which is because Streaming discards all middle context tokensw and retain a small number of initial attention sinks plus the most recent token. Tasks like NarrativeQA losing the middle portion directly hurts prediction quality. SnapKV generally gives a more favorable quality-memory trade off, since it tends to lose less quality whilst adding almost no compression overhead on top. H2O preserves quality well but is the slowest method due to requiring attention scores at every decoding step. Overall we find KV cache compression in this prototype setting to be most effective as a memory saving technique, with a smaller and more task-dependent latency benefit overall.

8 Discussion and Limitation

We find that KV cache compression is primarily useful as a memory-saving technique on our prototype. In all our tasks, smaller budgets yield smaller retained KV caches, and budgets of 0.5 often preserve task quality better than 0.2 budget. On the other hand, the latency benefit is more limited, and while we see small forward-pass gains in some settings, it does not always translate into clear end-to-end speedup, as total runtime contains compression overhead, prefill cost, and framework-level overhead. For a few cases, a more compressed setting is slightly better than a bigger budget or the full baseline, we interpret this to be sampled-subset variation and metric-level fluctuation rather than evidence that more compression is consistently better. For our study, consider the quality-memory trade-offs trade-offs we highlight relevant, and the latency conclusions considerably more limited in this Colab-based environment.

9 Conclusion

In this project, we compare several KV cache compression methods under an umbrella prototype benchmark. Our results ultimately show that KV cache compression can significantly decrease the retained KV cache size while often maintaining reasonable task quality at a 0.5 budget, but the speed benefit is smaller and less consistent than the memory benefit. Overall we find a quality-memory trade-off in KV cache compression.

10 Future Work

For future work, we may include evaluating more tasks, more samples, and longer contexts. We will also use more optimized implementations and cleaner GPU benchmark, which will help us to evaluate KV cache compression under settings where latency gains might be easier for us to observe.

References

Y. Bai et al. Longbench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019.

Y. Li et al. Snapkv: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

G. Xiao et al. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Manzil Zaheer et al. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020.

H. Zhang et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint*, 2023.

Z. Zhang et al. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.